

Tecnológico de Antioquia

2026

Prof Diego Botia

Guía de Laboratorio: Desarrollo de Aplicación Full Stack (Spring Boot + ReactJS)

1. Introducción

Este laboratorio tiene como objetivo guiar al estudiante en la creación de una aplicación web completa (Full Stack). Se utilizará **Java Spring Boot** para el desarrollo de una API RESTful que gestione la lógica de negocio y la persistencia de datos, y **ReactJS** para la interfaz de usuario. La aplicación permitirá realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre un catálogo de libros, utilizando una base de datos en memoria (H2) para facilitar las pruebas rápidas.

2. Objetivos

- Configurar un entorno de desarrollo para aplicaciones Full Stack.
- Desarrollar una API REST robusta utilizando Spring Boot y Spring Data JPA.
- Construir una interfaz de usuario dinámica y reactiva con ReactJS.
- Implementar la comunicación entre el frontend y el backend mediante peticiones HTTP (Axios).
- Configurar políticas de CORS (Cross-Origin Resource Sharing) para permitir la integración.

3. Arquitectura de la Aplicación

La aplicación sigue un modelo de arquitectura desacoplada:

- **Frontend (Cliente):** Desarrollado en ReactJS. Se encarga de la presentación y la interacción con el usuario. Utiliza componentes para la interfaz y servicios para comunicarse con la API.
- **Backend (Servidor):** Una aplicación Spring Boot que expone endpoints REST. Sigue un patrón de capas:
 - **Controlador:** Maneja las peticiones HTTP.
 - **Servicio:** Contiene la lógica de negocio.
 - **Repositorio:** Gestiona el acceso a los datos mediante JPA.
 - **Entidad:** Representa el modelo de datos (Libro).
- **Base de Datos:** H2 Database (In-memory), lo que significa que los datos se mantienen mientras la aplicación está en ejecución.

4. Procedimiento Paso a Paso

Parte A: Configuración del Backend (Spring Boot)

1. Generación del Proyecto

Utilice [Spring Initializr](#) con las siguientes especificaciones:

- **Project:** Maven
- **Language:** Java (v17 o superior)
- **Spring Boot:** 3.x.x
- **Dependencies:** Spring Web, Spring Data JPA, H2 Database, Lombok, Spring Boot DevTools.



Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Java

☐ Kotlin

☐ Groovy

☒ Maven

Spring Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (M3)

☐ 4.0.5 (SNAPSHOT)

☐ 4.0.4

☐ 3.5.13 (SNAPSHOT)

☒ 3.5.12

Project Metadata

Group

com.tdea

Artifact

book

Package name

com.tdea.book

Packaging

☒ Jar

☐ War

Configuration

☒ Properties

☐ YAML

Java

☐ 26

☐ 25

☐ 21

☒ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Boot DevTools

DEVELOPER TOOLS

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Lombok

DEVELOPER TOOLS

Java annotation library which helps to reduce boilerplate code.

Spring Web

WEB

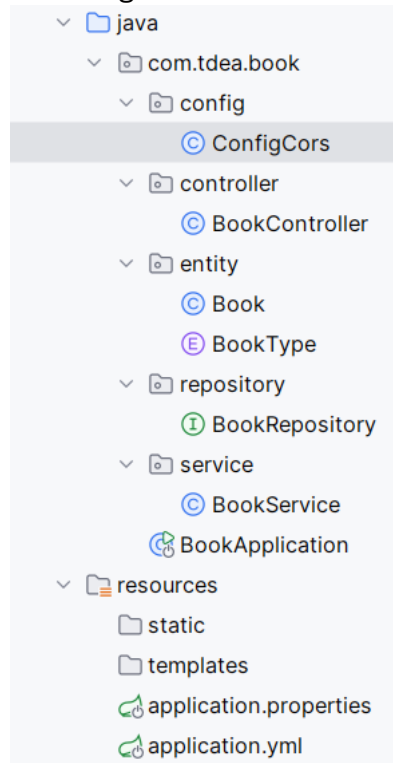
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

H2 Database

SQL

Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

Cuando cargue la aplicación de base en su IDE (por ejemplo IntelliJ) deje la siguiente estructura de paquetes para el proyecto



2. Configuración de la Base de Datos

En el archivo src/main/resources/application.yml (o .properties), configure el acceso a H2:

```
spring:
  datasource:
    url: jdbc:h2:mem:book
    driverClassName: org.h2.Driver
    username: sa
    password: sa
  jpa:
    database-platform: org.hibernate.dialect.H2Dialect
    hibernate:
      ddl-auto: update
  h2:
    console:
      enabled: true
```

3. Creación del Modelo y Capas

- **Entidad (Book):** Defina los atributos: id, name, isbn, publishedDate, price y bookType. Use anotaciones @Entity, @Id y @GeneratedValue.

Archivo BookType.java

```
package com.tdea.book.entity;

public enum BookType {
    EBOOK,SOFTCOPY,HARDCOVER
}
```

Archivo Book.java

```
package com.tdea.book.entity;

import jakarta.persistence.*;
import lombok.Builder;

import java.time.LocalDate;
```

Tecnológico de Antioquia

2026

Prof Diego Botia

@Builder //requerido para Book.builder()

@Entity

public class Book {

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 private Long id;

 private String name;

 private String isbnNumber;

 private LocalDate publishDate;

 private Double price;

 @Enumerated(EnumType.STRING)

 private BookType type;

 public Book() {

 }

 public Book(Long id, String name, String isbnNumber, LocalDate publishDate, Double price, BookType type) {

 this.id = id;

 this.name = name;

 this.isbnNumber = isbnNumber;

 this.publishDate = publishDate;

 this.price = price;

 this.type = type;

 }

 public Long getId() { return id; }

 public void setId(Long id) { this.id = id; }

 public String getName() { return name; }

 public void setName(String name) { this.name = name; }

 public String getIsbnNumber() { return isbnNumber; }

 public void setIsbnNumber(String isbnNumber) { this.isbnNumber = isbnNumber; }

 public LocalDate getPublishDate() { return publishDate; }

 public void setPublishDate(LocalDate publishDate) {

 this.publishDate = publishDate;

 }

 public Double getPrice() { return price; }

 public void setPrice(Double price) { this.price = price; }

 public BookType getType() { return type; }

 public void setType(BookType type) { this.type = type; }

}

- **Repositorio:** Cree una interfaz BookRepository que extienda de JpaRepository<Book, Long>.
- package com.tdea.book.repository;

import com.tdea.book.entity.Book;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;

@Repository

public interface BookRepository extends JpaRepository<Book, Long> {

}

- **Servicio:** Implemente BookService para manejar la lógica de búsqueda, guardado y eliminación.

```
package com.tdea.book.service;

import com.tdea.book.entity.Book;
import com.tdea.book.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class BookService {
    @Autowired
    private BookRepository bookRepository;

    public List<Book> findAll() {
        return bookRepository.findAll();
    }
    public Book findById(Long id) {
        return bookRepository.findById(id).get();
    }
    public Book save(Book book) {
        return bookRepository.save(book);
    }
    public void deleteById(Long id) {
        bookRepository.deleteById(id);
    }

    public Book update(Long id, Book book) {
        book.setId(id);
        return bookRepository.save(book);
    }
}
```

- **Controlador:** Cree BookController anotado con @RestController y @RequestMapping("/api/books"). Implemente los métodos @GetMapping, @PostMapping, @PutMapping y @DeleteMapping.

```
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping(value = "/api/books", produces = MediaType.APPLICATION_JSON_VALUE)
@Tag(name = "Book API", description = "CRUD Operations for Managing books")
public class BookController {

    private final BookService service;

    @Autowired
    public BookController(BookService service) {
        this.service = service;
    }
}
```

Tecnológico de Antioquia

2026

Prof Diego Botia

```
// ✅ GET ALL
@GetMapping
public ResponseEntity<List<Book>> getAllBooks() {
    return ResponseEntity.ok(service.findAll());
}

// ✅ GET BY ID (manejo de excepción)
@GetMapping("/{id}")
public ResponseEntity<Book> getBookById(@PathVariable Long id) {
    try {
        Book book = service.findById(id);
        return ResponseEntity.ok(book);
    } catch (Exception e) {
        return ResponseEntity.notFound().build(); // 404
    }
}

// ✅ CREATE
@PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Book> addBook(@Valid @RequestBody Book book) {

    Book savedBook = service.save(book);

    URI location = ServletUriComponentsBuilder
        .fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(savedBook.getId())
        .toUri();

    return ResponseEntity.created(location).body(savedBook); // 201
}

// ✅ UPDATE (sin Optional, validando existencia antes)
@PutMapping(value = "{id}", consumes = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Book> updateBook(@PathVariable Long id, @Valid @RequestBody Book book) {
    try {
        // Validar si existe primero
        service.findById(id);

        Book updatedBook = service.update(id, book);
        return ResponseEntity.ok(updatedBook); // 200

    } catch (Exception e) {
        return ResponseEntity.notFound().build(); // 404
    }
}

// ✅ DELETE
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteBook(@PathVariable Long id) {
    try {
        service.findById(id); // validar existencia
        service.deleteById(id);
        return ResponseEntity.noContent().build(); // 204
    } catch (Exception e) {
        return ResponseEntity.notFound().build(); // 404
    }
}
```

4. Configuración de CORS

Para permitir que React (puerto 3000) acceda a Spring Boot (puerto 8080), cree una clase de configuración:

```
Java
@Configuration
public class CorsConfig {
```

Tecnológico de Antioquia

2026

Prof Diego Botia

@Bean

```
public WebMvcConfigurer corsConfigurer() {
    return new WebMvcConfigurer() {
        @Override
        public void addCorsMappings(CorsRegistry registry) {
            registry.addMapping("/**")
                .allowedOrigins("http://localhost:3000")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS");
        }
    };
}
```

Parte B: Configuración del Frontend (ReactJS)

1. Inicialización del Proyecto

Abra una terminal y ejecute:

```
npx create-react-app catalog-frontend
```

```
cd catalog-frontend
```

```
npm install axios bootstrap
```

2. Estructura de Componentes

- **Servicio (BookService.js):** Cree una clase que utilice axios para realizar las peticiones a <http://localhost:8080/api/books>.
- **Componente BookList.jsx:** Use el hook useEffect para llamar al servicio y obtener la lista de libros al cargar la página. Renderice los datos en una tabla de Bootstrap.
- **Componente BookForm.jsx:** Cree un formulario para capturar los datos del nuevo libro (nombre, ISBN, fecha, etc.) y envíelos al backend mediante el método POST del servicio.

3. Integración en App.js

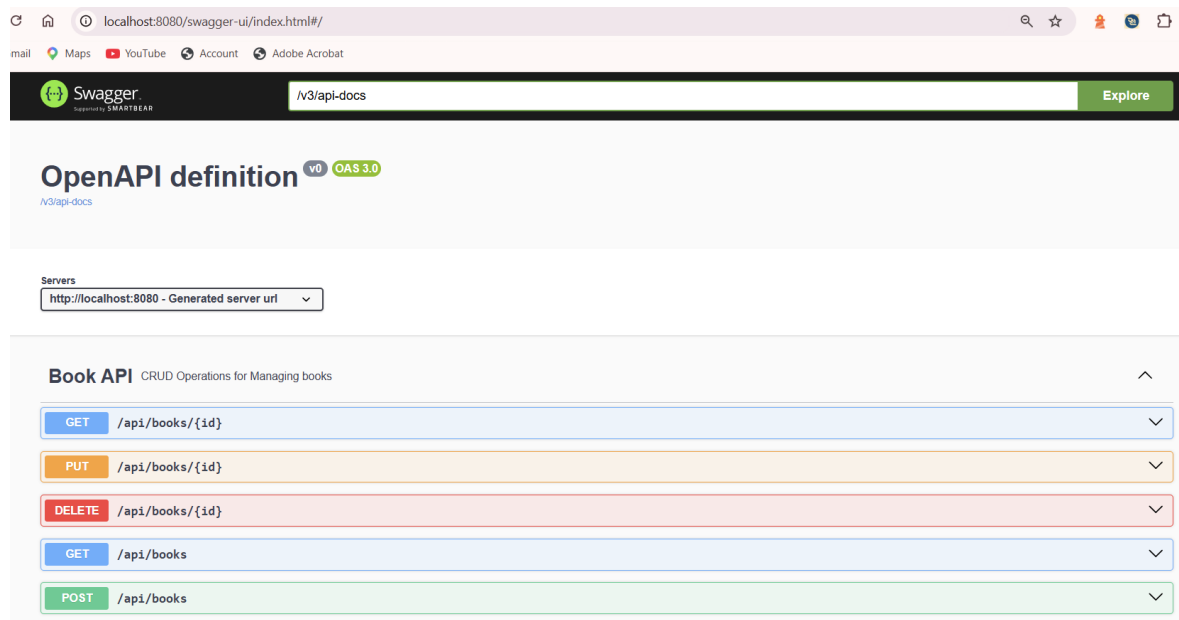
Importe y renderice ambos componentes en el archivo principal App.js para visualizar la gestión completa en una sola pantalla.

Parte C: Pruebas y Ejecución

1. **Ejecutar Backend:** Inicie la aplicación Spring Boot desde su IDE o usando `./mvnw spring-boot:run`. Verifique que el servidor corra en el puerto 8080.

Puede hacer pruebas de las APIs REST usando Open API o SWAGGER

Escriba en el navegador la siguiente ruta: <http://localhost:8080/swagger-ui.html> y pruebe cada endpoint.



2. **Ejecutar Frontend:** En la carpeta del proyecto React, ejecute `npm start`. Se abrirá el navegador en <http://localhost:3000>.
3. **Flujo de Trabajo:** * Agregue un libro mediante el formulario.
 - Observe cómo aparece automáticamente en la lista.

Tecnológico de Antioquia

2026

Prof Diego Botia

- Intente editar o eliminar un registro existente.
- (Opcional) Acceda a <http://localhost:8080/h2-console> para ver los datos directamente en la base de datos.

4. Entregables

FECHA DE ENTREGA: 18 de Mayo de 2026

1. Código fuente del Backend (Carpeta comprimida o enlace a repositorio).
2. Código fuente del Frontend. (Carpeta comprimida o enlace a repositorio).
3. Cree las vistas en REACT para que permita hacer operaciones CRUD y búsqueda de libros por nombre.
4. Breve informe con capturas de pantalla de la aplicación funcionando (Listado de libros y confirmación de creación/eliminación).